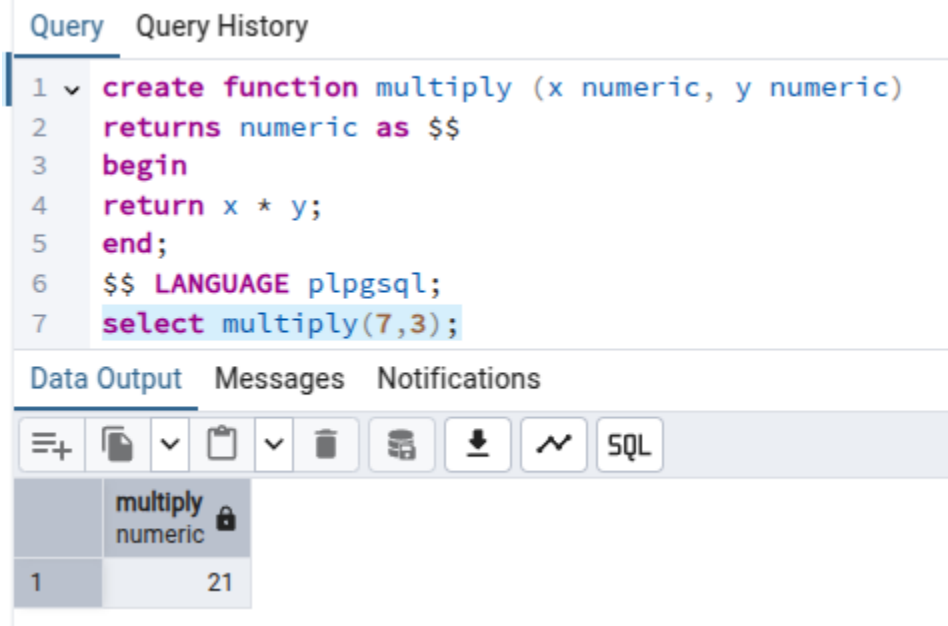


Lab 4

1-Create a multiply function that accepts two numbers and returns their product.



The screenshot shows a SQL IDE interface. At the top, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying a SQL script with line numbers 1 through 7. The script creates a function named 'multiply' that takes two numeric arguments, 'x' and 'y', and returns their product. The function is created using the 'CREATE FUNCTION' and 'LANGUAGE plpgsql' syntax. The final line of the script is a 'SELECT' statement that calls the 'multiply' function with arguments 7 and 3. Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with two columns: 'multiply' and 'numeric'. The table has one row with the value 21.

```
1 create function multiply (x numeric, y numeric)
2 returns numeric as $$
3 begin
4 return x * y;
5 end;
6 $$ LANGUAGE plpgsql;
7 select multiply(7,3);
```

	multiply	numeric
1		21

2-Create a hello_world function that takes a name as input and returns a personalized welcome message for that name.

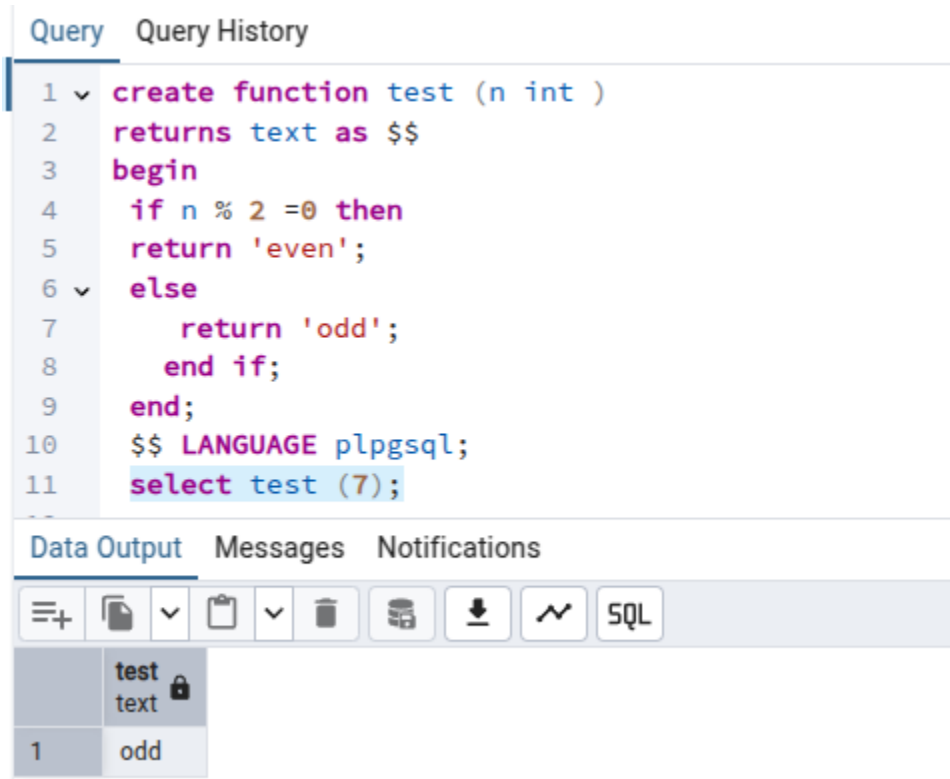


The screenshot shows a SQL IDE interface. At the top, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying a SQL script with line numbers 1 through 8. The script creates a function named 'hello_world' that takes a text argument, 'name', and returns a personalized welcome message. The function is created using the 'CREATE FUNCTION' and 'LANGUAGE plpgsql' syntax. The final line of the script is a 'SELECT' statement that calls the 'hello_world' function with the argument 'tadres'. Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with two columns: 'hello_world' and 'text'. The table has one row with the value 'welcome tadres'.

```
1 create function hello_world(name text)
2 returns text as $$
3 begin
4 return concat('welcome ', name);
5 end;
6 $$ LANGUAGE plpgsql;
7
8 select hello_world('tadres');
```

	hello_world	text
1		welcome tadres

3-Create a function that accepts a number and determines whether it is odd or even.



The screenshot shows a SQL IDE interface. At the top, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying a SQL script. The script defines a function 'test' that takes an integer 'n' as input and returns 'even' if 'n' is divisible by 2, and 'odd' otherwise. The function is created using PL/SQL. Below the script, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with one row and one column. The table has a header row with the column name 'test' and a data row with the value 'odd'.

```
1 create function test (n int )
2 returns text as $$
3 begin
4     if n % 2 =0 then
5         return 'even';
6     else
7         return 'odd';
8     end if;
9 end;
10 $$ LANGUAGE plpgsql;
11 select test (7);
```

	test
1	odd

4-Create a function that takes a Student ID as input and retrieves all information related to that student.

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

▼

INSERT INTO grades (stu_id, sub_id, exam_id, grade)

VALUES

(2, 1, 1, 88),

(3, 1, 1, 77);

▼

CREATE FUNCTION avg_grade_subject(p_sub_name TEXT)

RETURNS NUMERIC AS \$\$

DECLARE

avg_grade NUMERIC;

▼

BEGIN

SELECT AVG(g.grade) INTO avg_grade

FROM grades g

JOIN subject sub ON g.sub_id = sub.id

WHERE sub.sub_name = p_sub_name;

RETURN avg_grade;

END;

\$\$ LANGUAGE plpgsql;

SELECT avg_grade_subject('python programming');

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑️

🗑️

📥

⬇️

📈

SQL

avg_grade_subject

numeric

🔒

1

83.33333333333333

7-Create a trigger to monitor changes made to the student table, including additions, updates, and deletions. This trigger will record the time of each action and provide a description of the action in another table.

Query Query History

```
1 v create table logs(id serial primary key,opertion varchar(50),
2   action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
3 v CREATE function insert_log_data()
4   RETURNS TRIGGER as $$
5   BEGIN
6     if TG_OP = 'INSERT' then
7       insert into logs(opertion) values('you inserted new student');
8 v   elsif TG_OP = 'UPDATE' then
9     insert into logs(opertion) values(
10    concat('you updated student',NEW.id));
11 v   else
12     insert into logs(opertion) values('you delete student');
13   End if;
14   return NEW;
15 end;
16 $$language plpgsql;
17
18 v create TRIGGER stu_Logs
19 after insert or update or delete
20 on student
21 for each row
22 execute function insert_log_data();
23 select * from student;
24 v insert into student(e_name,email,address,track_id,birth_date,gender)
25 values('bebo','bebo@iti.com','assiut',1,'2001-09-07','Male');
26 select * from logs;
```

Data Output Messages Notifications

									SQL
	Id		opertion		action_time				
	[PK] integer		character varying (50)		timestamp without time zone				
1	1		you inserted new student		2025-05-01 20:43:37.898688				